

OOP Interview Questions

Բոլոր հնարավոր OOP հարցազրույցի հարցերը (C++ / Java)

Այս ֆայլը OOP-ի interview հարցերի ամբողջական ժողովածու է: Երեք մաս.

Մաս A — 40 multiple-choice հարց (4 տարբերակ, մեկը ճիշտ), կոդի output / concept հարցերով:

Մաս B — 50 բաց (open) տեսական հարց՝ առանց տարբերակների:

Մաս C — 20 tricky կոդային հարց (ի՞նչ կտալի / error է թե ոչ):

Բոլոր պատասխանները **միայն վերջի էջերում** են (Answer Key A, B, C): Առաջ նայելուց առաջ փորձիր ինքդ պատասխանել:

ՄԱՍ A — Multiple Choice (40)

Q1

OOP-ի 4 սյուններն զրոյն են: - (a) Class, Object, Method, Field - (b) Encapsulation, Inheritance, Polymorphism, Abstraction - (c) Public, Private, Protected, Default - (d) Compile, Runtime, Static, Dynamic

Q2

Ի՞նչ է class-ը: - (a) object-ի instance - (b) blueprint / template - (c) function - (d) variable

Q3

Ի՞նչ է object-ը: - (a) class-ի blueprint - (b) class-ի instance - (c) method - (d) interface

Q4

Encapsulation-ը հիմնականում ի՞նչ է անում: - (a) reuse code - (b) hide data - (c) hide implementation - (d) many forms

Q5

Inheritance-ը հիմնականում ի՞նչ է տալիս: - (a) data hiding - (b) code reuse - (c) hide implementation - (d) validation

Q6

Polymorphism-ը մեկ նախադասությամբ. - (a) hide data - (b) reuse code - (c) one interface, many forms - (d) hide implementation

Q7

Abstraction-ը ի՞նչ է անում: - (a) hide implementation, show essentials - (b) reuse code - (c) protect data with private - (d) create objects

Q8

Runtime polymorphism-ը C++-ում ինչո՞վ է իրականացվում: - (a) function overloading - (b) operator overloading - (c) virtual functions - (d) templates

Q9

Compile-time polymorphism-ի օրինակ. - (a) virtual function - (b) function/operator overloading - (c) abstract class - (d) dynamic_cast

Q10

Pure virtual function-ի syntax-ը C++-ում. - (a) `virtual void f();` - (b) `void f() = 0;` - (c) `virtual void f() = 0;` - (d) `abstract void f();`

Q11

Class-ը abstract է դառնում, երբ. - (a) ունի private member - (b) ունի գոնե մեկ pure virtual function - (c) ունի static member - (d) ունի constructor

Q12

Abstract class-ից object. - (a) կարելի է ստեղծել - (b) չի կարելի ստեղծել - (c) միայն pointer-ով - (d) միայն static

Q13

Ինչո՞ւ է virtual destructor-ը պետք: - (a) արագացնում է - (b) base pointer-ով delete անելիս derived destructor-ն էլ կանչվի - (c) memory խնայում է - (d) պարտադիր է ամեն class-ում

Q14

Java-ում multiple inheritance class-ներով. - (a) թույլատրվում է - (b) արգելված է (Diamond Problem) - (c) միայն 2 class-ով - (d) միայն abstract-ով

Q15

C++-ում Diamond Problem-ը լուծվում է. - (a) interface-ով - (b) virtual inheritance-ով - (c) friend-ով - (d) static-ով

Q16

Overloading-ը երբ է որոշվում. - (a) runtime - (b) compile-time - (c) link-time - (d) երբեք

Q17

Overriding-ը երբ է որոշվում. - (a) compile-time - (b) runtime - (c) load-time - (d) երբեք

Q18

private member-ը հասանելի է. - (a) միայն նույն class-ում - (b) child class-ում - (c) նույն package-ում - (d) ամեն տեղից

Q19

protected member-ը հասանելի է. - (a) միայն նույն class-ում - (b) նույն class + child class + (Java) նույն package - (c) միայն child class - (d) ամեն տեղից

Q20

Constructor-ի հերթականությունը inheritance-ում. - (a) Derived -> Base - (b) Base -> Derived - (c) միաժամանակ - (d) պատահական

Q21

Destructor-ի հերթականությունը inheritance-ում. - (a) Base -> Derived - (b) Derived -> Base - (c) միաժամանակ - (d) պատահական

Q22

"Dog is an Animal" կապը. - (a) has-a (composition) - (b) is-a (inheritance) - (c) uses-a - (d) friend

Q23

"Car has an Engine" կապը. - (a) is-a (inheritance) - (b) has-a (composition) - (c) is-like-a - (d) static

Q24

Interview-ի ընդհանուր խորհուրդը. - (a) favor inheritance over composition - (b) favor composition over inheritance - (c) միշտ multiple inheritance - (d) երբեք չօգտագործել virtual

Q25

Abstract class-ը C++-ում կարող է ունենալ սովորական (non-pure) մեթոդներ. - (a) ոչ - (b) այո - (c) միայն static - (d) միայն private

Q26

Interface-ը (Java) կարող է ունենալ state (instance fields): - (a) այո, ցանկացած - (b) միայն **public static final** (constant) - (c) միայն private - (d) ոչ մի field

Q27

Ո՞ր operator-ը ՉԻ կարելի overload անել C++-ում. - (a) `+` - (b) `[]` - (c) `::` - (d) `<<`

Q28

`<<` operator-ը (stream) սովորաբար. - (a) member function - (b) non-member (friend) - (c) static member - (d) virtual

Q29

Postfix `operator++(int)` -ի `int` parameter-ը ինչի՞ համար է. - (a) արժեք ավելացնելու - (b) prefix-ից տարբերելու համար (dummy) - (c) index-ի - (d) ոչնչի

Q30

Prefix `++` -ը սովորաբար վերադարձնում է. - (a) նոր object by value - (b) reference (`*this`) - (c) void - (d) bool

Q31

`static` member-ը պատկանում է. - (a) ամեն object-ին առանձին - (b) class-ին (մեկ instance) - (c) main-ին - (d) ոչ մեկին

Q32

`this` pointer-ը ի՞նչ է. - (a) base class-ի հասցե - (b) ընթացիկ object-ի հասցե - (c) static member - (d) vtable

Q33

Rule of Three-ը վերաբերում է. - (a) 3 constructor - (b) destructor, copy constructor, copy assignment - (c) 3 inheritance level - (d) 3 pillar

Q34

`friend` function-ը. - (a) ժառանգվում է - (b) հասանելիություն ունի private անդամներին - (c) virtual է - (d) static է

Q35

Java-ում virtual by default. - (a) ոչ մի մեթոդ - (b) բոլոր non-static մեթոդները - (c) միայն abstract - (d) միայն static

Q36

C++-ում մեթոդը virtual դառնում է. - (a) ավտոմատ - (b) `virtual` keyword-ով - (c) `static` -ով - (d) `friend` -ով

Q37

SOLID-ի "S"-ը նշանակում է. - (a) Static - (b) Single Responsibility - (c) Simple - (d) Standard

Q38

Dependency Inversion-ը ասում է. - (a) կախված եղիր concrete class-ից - (b) կախված եղիր abstraction-ից - (c) մի օգտագործիր interface - (d) միշտ inheritance

Q39

Method overriding-ի համար C++-ում պահանջվում է. - (a) նույն անուն, տարբեր parameters - (b) նույն signature + virtual base-ում - (c) static մեթոդ - (d) friend

Q40

Composition-ի հիմնական առավելությունը. - (a) ավելի արագ compile - (b) flexibility + less coupling - (c) multiple inheritance - (d) data hiding

ՄԱՍ B — Open Questions (50)

Բաց հարցեր, փորձիր ամբողջական պատասխանել ինչպես interview-ում: Մոդելային պատասխանները՝ Answer Key B-ում:

1. Ի՞նչ է OOP-ը և ինչո՞վ է տարբերվում procedural programming-ից:
2. Ի՞նչ է class-ը և object-ը, բեր օրինակ:
3. Բացատրիր encapsulation-ը և իր 4 առավելությունը:
4. Ի՞նչ է data hiding-ը, կապը encapsulation-ի հետ:
5. Բացատրիր inheritance-ը և իր առավելությունները:
6. Թվարկիր inheritance-ի տեսակները:
7. Ի՞նչ է polymorphism-ը, ի՞նչ տեսակներ կան:
8. Overloading vs overriding — մանրամասն տարբերությունը:
9. Ի՞նչ է abstraction-ը, ինչո՞վ է տարբերվում encapsulation-ից:
10. Abstract class vs interface — համեմատիր:
11. Ի՞նչ է pure virtual function-ը:
12. Ի՞նչ է virtual function-ը, ինչպես է աշխատում (vtable/vptr):
13. Ինչո՞ւ է virtual destructor-ը պետք, ի՞նչ է լինում առանց դրա:
14. Ի՞նչ է Diamond Problem-ը, ինչպես է լուծվում Java-ում և C++-ում:
15. Ինչո՞ւ Java-ում class-ներով multiple inheritance չկա:
16. Բացատրիր access modifier-ները (private/protected/public/default):
17. Private member-ը ժառանգվո՞ւմ է:
18. Ի՞նչ է constructor-ը, ի՞նչ տեսակներ կան:
19. Ի՞նչ է destructor-ը, երբ է կանչվում:
20. Constructor/destructor-ի հերթականությունը inheritance-ում:
21. Ի՞նչ է copy constructor-ը:
22. Shallow copy vs deep copy:
23. Ի՞նչ է Rule of Three (և Rule of Five):
24. Ի՞նչ է move constructor-ը (C++):
25. Composition vs Inheritance — երբ որն օգտագործել:
26. Ի՞նչ է aggregation-ը, ինչո՞վ է տարբերվում composition-ից:
27. Ի՞նչ է association-ը:
28. Ի՞նչ է `this` pointer-ը:
29. Ի՞նչ է static member/method-ը:
30. Ինչո՞ւ static մեթոդը չի կարող non-static member օգտագործել:
31. Ի՞նչ է `friend` function/class-ը:
32. Ի՞նչ է operator overloading-ը, բեր օրինակ:
33. Ո՞ր operator-ները չեն overload-վում:
34. `<<` operator-ը ինչո՞ւ պետք է non-member լինի:

35. Prefix vs postfix `++` overload-ի տարբերությունը:
 36. Ի՞նչ է functor-ը (`operator()`):
 37. Ի՞նչ է `const` member function-ը:
 38. Ի՞նչ է `virtual` և `override` keyword-ների տարբերությունը:
 39. Ի՞նչ է `final` keyword-ը OOP-ում:
 40. Բացատրիր SOLID-ի 5 սկզբունքները:
 41. Ի՞նչ է Single Responsibility Principle-ը:
 42. Ի՞նչ է Open/Closed Principle-ը:
 43. Ի՞նչ է Liskov Substitution Principle-ը:
 44. Ի՞նչ է Dependency Injection/Inversion-ը:
 45. Ի՞նչ է coupling և cohesion-ը:
 46. Ի՞նչ է object slicing-ը (C++):
 47. Constructor-ը կարող է virtual լինել: Ինչու:
 48. Ի՞նչ է abstract method vs concrete method-ը:
 49. C++ vs Java OOP — հիմնական տարբերությունները:
 50. Ի՞նչ է `is-a` , `has-a` , և `uses-a` կապերը:
-

U&U C — Tricky Code (20)

C1

```
class A { public: void f() { cout << "A"; } };  
class B : public A { public: void f() { cout << "B"; } };  
A* p = new B();  
p->f();
```

- (a) A (static binding, virtual չկա)
- (b) B
- (c) compile error
- (d) AB

C2

```
class A { public: virtual void f() { cout << "A"; } };  
class B : public A { public: void f() override { cout << "B"; } };  
A* p = new B();  
p->f();
```

- (a) A
- (b) B
- (c) CE
- (d) undefined

C3

```
class A { public: virtual void f() = 0; };  
A a;
```

- (a) OK
- (b) compile error (abstract class object)
- (c) runtime error
- (d) ոչ ինչ

C4

```
class A {
public:
    A() { cout << "A "; }
    ~A() { cout << "~A "; }
};
class B : public A {
public:
    B() { cout << "B "; }
    ~B() { cout << "~B "; }
};
B obj;
```

- (a) A B ~B ~A
- (b) B A ~A ~B
- (c) A B ~A ~B
- (d) B A ~B ~A

C5

```
class A {
public:
    ~A() { cout << "A"; }           // not virtual
};
class B : public A {
public:
    ~B() { cout << "B"; }
};
A* p = new B();
delete p;
```

- (a) BA
- (b) A (միայն, non-virtual dtor)
- (c) AB
- (d) B

C6

```
class A {  
    int x;  
public:  
    void set(int x) { x = x; }           // ուշադրություն  
};
```

`set(5)` -ից հետո member `x` -ն ինչ է: - (a) 5 - (b) չի փոխվում (parameter-ը ինքն իրեն է assign, `this->x` պետք էր) - (c) 0 - (d) CE

C7

```
class A { public: void f() { cout << "A"; } };  
class B : private A { };  
B b;  
b.f();
```

- (a) A
- (b) compile error (private inheritance, f() հասանելի չէ դրսից)
- (c) B
- (d) ոչինչ

C8

```
class Counter {  
    int c = 0;  
public:  
    Counter& operator++() { ++c; return *this; }  
    int get() { return c; }  
};  
Counter x;  
++(++x);  
cout << x.get();
```

- (a) 1
- (b) 2
- (c) 0
- (d) CE

C9

```
class A {  
public:  
    static int count;  
    A() { count++; }  
};  
int A::count = 0;  
A a1, a2, a3;  
cout << A::count;
```

- (a) 1
- (b) 3
- (c) 0
- (d) CE

C10

```
class A {  
public:  
    virtual void f() { cout << "A"; }  
};  
class B : public A {  
public:  
    void f() { cout << "B"; }  
};  
B b;  
A a = b;    // object slicing  
a.f();
```

- (a) B
- (b) A (slicing -> միայն A մասը)
- (c) CE
- (d) undefined

C11

```
class A {  
public:  
    A() { f(); } // constructor-ի ներսում virtual call  
    virtual void f() { cout << "A"; }  
};  
class B : public A {  
public:  
    void f() override { cout << "B"; }  
};  
B b;
```

- (a) B
- (b) A (ctor-ի ներսում virtual -> ընթացիկ class)
- (c) CE
- (d) AB

C12

```
class A {  
public:  
    int add(int a, int b) { return a + b; }  
    double add(int a, int b) { return a + b; } // միայն return type տարբեր  
};
```

- (a) OK (overload)
- (b) compile error (return type-ը overload չի սահմանում)
- (c) runtime error
- (d) ambiguous call

C13

```
class Base {
public:
    void show() { cout << "Base"; }
};
class Derived : public Base {
public:
    void show() { cout << "Derived"; }
};
Derived d;
d.Base::show();
```

- (a) Derived
- (b) Base (explicit scope)
- (c) CE
- (d) BaseDerived

C14

```
class A {
public:
    virtual void f(int) { cout << "A"; }
};
class B : public A {
public:
    void f(double) { cout << "B"; }    // unapplied signature
};
A* p = new B();
p->f(5);
```

- (a) A (B::f(double) override ȳ, name hiding)
- (b) B
- (c) CE
- (d) undefined

C15

```
class A {  
public:  
    A(int x) { cout << x; }  
};  
A a;
```

- (a) 0
- (b) compile error (default constructor չկա)
- (c) garbage
- (d) ոչինչ

C16

```
class Point {  
public:  
    int x;  
    Point(int x) : x(x) {}  
    bool operator==(const Point& p) const { return x == p.x; }  
};  
Point a(5), b(5);  
cout << (a == b);
```

- (a) 0
- (b) 1
- (c) CE
- (d) undefined

C17

```
class A {  
public:  
    void f() const { cout << "const"; }  
    void f()       { cout << "non-const"; }  
};  
const A a;  
a.f();
```

- (a) const

- (b) non-const
- (c) ambiguous CE
- (d) undefined

C18

```
class Base {
public:
    virtual void f() final { cout << "Base"; }
};

class Derived : public Base {
public:
    void f() { cout << "Derived"; }    // override final
};
```

- (a) OK
- (b) compile error (cannot override final)
- (c) runtime error
- (d) Derived

C19

```
class A {  
public:  
    A& operator=(const A& o) {  
        cout << "assign";  
        return *this;  
    }  
};  
  
A a, b;  
a = b;
```

- (a) assign
- (b) $n_2[h_2]$ (copy constructor $\downarrow$)
- (c) CE
- (d) assignassign

C20

```
class A { public: int x = 1; };  
class B { public: int x = 2; };  
class C : public A, public B { };  
C c;  
cout << c.x;
```

- (a) 1
 - (b) 2
 - (c) compile error (ambiguous, A::x vs B::x)
 - (d) 3
-

ANSWER KEY A (Multiple Choice)

1-b · 2-b · 3-b · 4-b · 5-b · 6-c · 7-a · 8-c · 9-b · 10-c 11-b · 12-b · 13-b · 14-b · 15-b · 16-b · 17-b · 18-a ·
19-b · 20-b 21-b · 22-b · 23-b · 24-b · 25-b · 26-b · 27-c · 28-b · 29-b · 30-b 31-b · 32-b · 33-b · 34-b · 35-
b · 36-b · 37-b · 38-b · 39-b · 40-b

ANSWER KEY B (Open Questions)

1. OOP-ը ծրագրավորման paradigm է object-ների ու class-ների շուրջ: Procedural-ը կենտրոնացնում է ֆունկցիաների/քայլերի վրա, OOP-ը՝ տվյալների ու վարքի միավորման (object) վրա, ինչը տալիս է modularity, reuse, maintainability.
2. Class-ը blueprint/template է (սահմանում է դաշտեր ու մեթոդներ); object-ը class-ի instance-ն է հիշողությունում. `class Car {...}; Car c;` — `c`-ն object է.
3. Encapsulation = տվյալների ու մեթոդների միավորում մեկ class-ում + ուղիղ հասանելիության սահմանափակում (private + getter/setter). Առավելություններ՝ data hiding, security, better control (validation), easy maintenance.
4. Data hiding = ներքին տվյալները թաքցնելը արտաքին կողմից (`private`), որ դրանք փոփոխվեն միայն վերահսկված մեթոդներով. encapsulation-ի հիմնական արդյունքն է.
5. Inheritance = մեկ class ստանում է մյուսի հատկություններն ու մեթոդները (`extends` / `:` `public`). Առավելություններ՝ code reuse, reduced duplication, easier maintenance, polymorphism-ի հիմք.
6. Single, Multilevel, Hierarchical, Multiple (Java-ում interface-ով), Hybrid.
7. Polymorphism = «շատ ձև». Compile-time (overloading `function/operator`) և Runtime (`overriding` virtual functions).
8. Overloading `սույն անուն տարբեր parameters, compile-time, inheritance պետք չէ`. `Overriding` նույն signature derived class-ում, runtime, պահանջում է inheritance + (C++) virtual.
9. Abstraction = թաքցնել implementation-ը, ցույց տալ միայն էականը (abstract class/interface). Encapsulation-ը data-ն է թաքցնում (private); abstraction-ը complexity/implementation-ը.
10. Abstract class `կարող է ունենալ implementation + state + constructor, single inheritance`. `Interface` մաքուր contract, multiple inheritance, (Java 8+) default/static մեթոդներ, միայն constant fields.
11. Pure virtual (`virtual void f() = 0;`) — implementation չունեցող virtual, որ derived-ը պարտավոր է override անել; class-ը դարձնում abstract.
12. Virtual function-ը թույլ է տալիս runtime dispatch. Compiler-ը ստեղծում է per-class vtable (function հասցեներ) և per-object vptr (ցույց է տալիս vtable-ին). Call-ը՝ object -> vptr -> vtable -> իրական մեթոդ.
13. Base pointer-ով derived object delete անելիս, եթե dtor virtual չէ, միայն base dtor-ը կկանչվի -> derived-ի resource-ները չեն ազատվի -> memory leak / UB.

14. Diamond Problem երբ class-ը երկու ճանապարհով ժառանգում է նույն base-ը -> ambiguity. Java class multiple inheritance արգելված, interface-ով + override. C++ virtual inheritance (base-ը մեկ instance).
15. Diamond Problem-ի ambiguity-ից խուսափելու համար; interface-ները (state չունեն) այդ խնդիրը չեն ստեղծում.
16. private միայն նույն class; default (Java) նույն package; protected class + child + (Java) package; public ամեն տեղից.
17. Object-ի հիշողությունում կա, բայց child class-ը ուղիղ չի կարող օգտագործել; հասանելի է միայն public getter-ով.
18. Constructor object ստեղծելիս ավտոմատ կանչվող մեթոդ, initialize է անում. Տեսակներ default, parameterized, copy, (C++11) move.
19. Destructor object ոչնչանալիս ավտոմատ կանչվում, cleanup է անում. Stack scope-ից դուրս; heap delete`-ի ժամանակ.
20. Ctor Base -> Derived. Dtor Derived -> Base (հակառակ).
21. Copy constructor նոր object ստեղծում գոյություն ունեցող-ից. A(const A& o)`.
22. Shallow միայն pointer/հասցե պատճենում (նույն memory). Deep նոր memory + բովանդակության պատճեն (անկախ object-ներ).
23. Rule of Three եթե պետք ունես destructor/copy-ctor/copy-assignment-ից մեկը, հավանաբար պետք ունես երեքն էլ (raw resource-ի պատճառով). Rule of Five + move ctor & move assignment (C++11).
24. Move constructor A(A&& o)` — resource-ի ownership է փոխանցում copy-ի փոխարեն, ավելի արագ մեծ object-ների համար.
25. Inheritance իրական "is-a" կապ (Dog is Animal). Composition "has-a" (Car has Engine). Favor composition` ճկունություն + քիչ coupling.
26. Aggregation weak has-a, մասը կարող է առանձին ապրել (Department has Teachers). Composition strong, մասը ամբողջի հետ մեռնում (House has Rooms).
27. Association` երկու անկախ object-ների ընդհանուր կապ (Teacher - Student), առանց ownership-ի.
28. `this`` ընթացիկ object-ի հասցեն, հասանելի ամեն non-static մեթոդում; օգտագործվում parameter/member տարբերելու և chaining-ի համար.
29. static member/method պատկանում class-ին (ոչ object-ին), մեկ instance բոլորի համար; կանչվում Class::member`-ով, object պետք չէ.
30. static մեթոդը this չունի (object չկա), ուստի չի կարող դիմել non-static member-ին, որ կոնկրետ object է պահանջում.
31. friend`` function/class, որ հասանելիություն ունի class-ի private/protected անդամներին; խախտում է encapsulation-ը վերահսկված ձևով (օր. operator<<`).

32. Operator overloading `operator`-ին տալ իմաստ `user type`-ի համար, օր. `Point operator+(const Point&) . p3 = p1 + p2``.
33. `::` (scope), `.` (member access), `.*` (member pointer), `?:` (ternary), `sizeof`, `typeid`.
34. `cout << p` -ում ձախ operand-ը `ostream` է (ոչ `pn` class), ուստի member չի կարող լինել; non-member (հաճախ friend), վերադարձնում `ostream&` chaining-ի համար.
35. Prefix (`++x`) փոխում է ու վերադարձնում `*this` (reference). Postfix (`x++`) ունի dummy `int` parameter, պահում հին արժեքը, վերադարձնում by value (ավելի թանկ).
36. Functor `class` `operator()` -ով, որ դարձնում է object-ը «կանչելի» ֆունկցիայի պես; STL-ում (comparator, transform) շատ օգտագործվող.
37. `const` member function (`T f() const`) երաշխավորում է, որ մեթոդը object-ը չի փոփոխի; `const` object-ների վրա միայն `const` մեթոդ կարելի է կանչել.
38. `virtual` base`-ում հայտարարում է, որ մեթոդը կարող է override լինել (runtime dispatch). `override` derived`-ում ստուգում է, որ իրոք override է անում (compiler բռնում սխալ signature).
39. `final``` արգելում է հետագա override (մեթոդի վրա) կամ inheritance (class-ի վրա).
40. S Single Responsibility (մեկ `class` մեկ պատասխանատվություն); O Open/Closed (բաց ընդլայնման, փակ փոփոխման); L Liskov (child փոխարինի parent-ին); I Interface Segregation (փոքր interface-ներ); D Dependency Inversion (կախված abstraction-ից).
41. Class-ը պետք ունենա փոփոխման միայն մեկ պատճառ (մեկ պատասխանատվություն); օր. `Invoice` -ը չպետք է և հաշվի, և տպի, և պահի DB-ում.
42. Software entities-ը պետք լինեն բաց ընդլայնման, բայց փակ փոփոխման; նոր վարք ավելացրու նոր կոդով (inheritance/interface), ոչ հին կոդը փոխելով.
43. Derived object-ը պետք կարողանա փոխարինել base-ին առանց ծրագրի ճիշտությունը կոտրել; օր. `Square extends Rectangle` -ը կարող է խախտել Liskov-ը.
44. Dependency Inversion կախված եղիք abstraction-ից, ոչ concrete class-ից. `Dependency Injection` dependency-ն դրսից տալ (constructor/setter), ոչ ներսում ստեղծել` testability-ի ու ճշգրտության համար.
45. Coupling որքան մի module կախված է մյուսից (weak coupling լավ). Cohesion որքան մի module-ի ներսի elementery `kapvac` են մեկ նպատակի (բարձր cohesion լավ).
46. Object slicing երբ derived object-ը վերագրում են base object-ի (ոչ pointer/reference), derived մասը «կտրվում» է, մնում միայն base-ը. `Base b = derivedObj;`.
47. Ոչ. Constructor-ի պահին `vptr`-ը դեռ set չէ, և object-ը դեռ ամբողջությամբ չկա, ուստի virtual dispatch իմաստ չունի.
48. Abstract method implementation չունի (pure virtual/abstract), derived-ը override է անում. Concrete method ունի implementation.

49. C++ multiple inheritance (virtual), virtual պեղք է հայտարարել, operator overloading, ձեռքով memory (new/delete), destructor. Java միայն interface multiple, բոլորն virtual by default, no operator overloading, GC, no destructor.

50. is-a inheritance (Dog is Animal); has-a composition/aggregation (Car has Engine); uses-a` dependency (մի class ժամանակավոր օգտագործում է մյուսին, օր. parameter-ով).

ANSWER KEY C (Tricky Code)

C1-a · C2-b · C3-b · C4-a · C5-b · C6-b · C7-b · C8-b · C9-b · C10-b · C11-b · C12-b · C13-b · C14-a · C15-b · C16-b · C17-a · C18-b · C19-a · C20-c

Բացատրutyunner (C)

C1 virtual չկա -> static binding -> pointer-ի տիպը A է -> "A". **C2** virtual + override -> dynamic dispatch -> "B". **C3** abstract class-ից object -> compile error. **C4** ctor A->B, dtor B->A -> "A B ~B ~A". **C5** non-virtual dtor + base ptr delete -> միայն "A". **C6** x = x parameter-ը ինքն իրեն, member չի փոխվում; պետք էր this->x = x. **C7** private inheritance -> f() դրսից հասանելի չէ -> CE. **C8** ++(++x) -> 2 (prefix reference-ի շնորհիվ chain). **C9** static count 3 object -> 3. **C10** object slicing (A a = b) -> միայն A մասը -> "A". **C11** ctor-ի ներքոնում virtual call -> ընթացիկ class (A) -> "A". **C12** միայն return type warber -> overload չէ -> CE. **C13** d.Base::show() explicit scope -> "Base". **C14** f(double) -ը override չէ (signature warber), name hiding; p->f(5) base-ի vtable -> A::f(int) -> "A". **C15** parameterized ctor կա, default չկա -> A a; CE. **C16** operator== -> 5==5 -> true -> 1. **C17** const object -> const overload -> "const". **C18** final մեթոդ override -> CE. **C19** a = b -> copy assignment operator -> "assign". **C20** c.x ambiguous (A::x vs B::x) -> CE (պետք է c.A::x`).

Ընդհանուր կanonner (հիշիր interview-ին)

- virtual չկա -> static binding (pointer-ի տիպ)
- virtual կա -> dynamic dispatch (object-ի իրական տիպ)
- ctor: Base->Derived | dtor: Derived->Base
- non-virtual dtor + base ptr delete -> leak
- ctor/dtor-ի ներքոնում virtual -> ընթացիկ class
- object slicing: Base b = derived -> միայն base մասն
- overload = signature warber (return type չհաավարդ)
- override = նույն signature + virtual
- final -> չի override/inherit
- multiple base նույն member -> ambiguous (scope պետք)